600 preguntas para construir criterio de Java Backend

Java | JVM | Maven | Spring | Spring Boot | JPA/Hibernate | SQL | REST | Testing | Clean Code | Arquitectura | Producción

No es una lista para memorizar. Es un banco de preguntas para obligarte a explicar cómo funciona el backend, justificar decisiones y detectar lagunas.

Cómo usarlo: responde primero sin consultar. Si no puedes explicar el “por qué”, marca la pregunta y vuelve después con documentación, código o experimentos.

Regla práctica de autoevaluación

Nivel	Qué deberías poder hacer	Señal de dominio
1 - No sé	No puedes explicar el concepto ni dar un ejemplo.	Debes estudiar y experimentar.
2 - Parcial	Puedes definirlo, pero dudas al justificar trade-offs o edge cases.	Debes practicar explicándolo y escribir código.
3 - Puedo explicarlo	Puedes explicarlo, compararlo y relacionarlo con Spring/Java.	El concepto forma parte de tu caja de herramientas.
4 - Puedo diseñarlo	Puedes aplicarlo a un caso real, anticipar fallos y justificar decisiones.	Nivel de criterio técnico que se espera de un backend sólido.

Importante: no existe una lista universal que “certifique” que alguien es Java Backend. Este banco está diseñado como una cobertura amplia de conocimientos y criterio técnico.

Mapa del banco

01. Java: lenguaje, OOP y fundamentos - 65 preguntas
02. Collections, Generics y programación funcional - 40 preguntas
03. JVM, memoria, concurrencia y rendimiento - 55 preguntas
04. Maven, dependencias y build - 30 preguntas
05. Spring Core: IoC, DI, Beans y AOP - 45 preguntas
06. Spring Boot, Web, MVC y configuración - 50 preguntas
07. JPA, Hibernate y persistencia - 55 preguntas
08. SQL, bases de datos y modelado de datos - 30 preguntas
09. REST, HTTP, seguridad y diseño de APIs - 30 preguntas
10. Testing: unit, integration, web y calidad - 25 preguntas
11. Clean Code, SOLID y diseño - 25 preguntas
12. Arquitectura, DDD, microservicios y sistemas distribuidos - 30 preguntas

13. Producción: observabilidad, Docker, CI/CD y operación - 20 preguntas

14. Seguridad: secure coding, OWASP, identidad, APIs, criptografía y DevSecOps - 100 preguntas

01. Java: lenguaje, OOP y fundamentos

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

001. **[Fundamentos]** ¿Qué problema resuelve la encapsulación y cómo la aplicarías en una clase de dominio real?

Tu respuesta: _____ _____

002. **[Fundamentos]** ¿Cuál es la diferencia entre una clase y un objeto? Da un ejemplo que no sea académico.

Tu respuesta: _____ _____

003. **[Fundamentos]** ¿Qué significa que Java sea de tipado estático y qué ventajas y costes implica?

Tu respuesta: _____ _____

004. **[Fundamentos]** ¿Qué diferencia hay entre tipo primitivo y tipo referencia? ¿Qué cambia en memoria y en comportamiento?

Tu respuesta: _____ _____

005. **[Fundamentos]** ¿Por qué `String` es inmutable y qué consecuencias tiene eso para un backend?

Tu respuesta: _____ _____

006. **[Fundamentos]** ¿Qué significa que Java pase argumentos por valor? Demuéstralo con un objeto mutable y con un primitivo.

Tu respuesta: _____ _____

007. **[Fundamentos]** ¿Qué diferencia conceptual existe entre identidad y estado de un objeto?

Tu respuesta: _____ _____

008. **[Fundamentos]** ¿Cuándo usarías `==` y cuándo `equals()`? ¿Qué error producirías al confundirlos?

Tu respuesta: _____ _____

009. **[Fundamentos]** ¿Qué contrato deben respetar `equals()` y `hashCode()` y por qué importa en `HashMap`/`HashSet`?

Tu respuesta: _____ _____

010. **[Fundamentos]** ¿Qué es la herencia y cuándo puede introducir un acoplamiento difícil de mantener?

Tu respuesta: _____ _____

011. **[Fundamentos]** ¿Por qué composición sobre herencia es una recomendación frecuente en diseño orientado a objetos?

Tu respuesta: _____ _____

012. **[Fundamentos]** ¿Qué diferencia hay entre clase abstracta e interfaz? Explica cómo decidir entre ambas.

Tu respuesta: _____ _____

013. **[Fundamentos]** ¿Qué significa polimorfismo en Java? Explica la diferencia entre polimorfismo estático y dinámico.

Tu respuesta: _____ _____

014. **[Fundamentos]** ¿Qué ocurre cuando haces overriding de un método? ¿Cómo decide Java qué implementación ejecutar?

Tu respuesta: _____ _____

015. **[Fundamentos]** ¿Qué diferencia hay entre overloading y overriding y qué problemas aparecen si se confunden?

Tu respuesta: _____ _____

016. **[Fundamentos]** ¿Qué reglas aplican a la visibilidad al hacer overriding de un método?

Tu respuesta: _____ _____

017. [Fundamentos] ¿Qué son los modificadores `public`, `protected`, package-private y `private`?

Tu respuesta: _____ _____

018. [Fundamentos] ¿Qué significa que una clase sea `final`? ¿Cuándo lo usarías deliberadamente?

Tu respuesta: _____ _____

019. [Fundamentos] ¿Qué significa que un método sea `static`? ¿Qué relación tiene con instancias y estado?

Tu respuesta: _____ _____

020. [Fundamentos] ¿Por qué abusar de métodos estáticos puede dificultar testing y diseño?

Tu respuesta: _____ _____

021. [Fundamentos] ¿Qué hace un constructor y qué responsabilidad debería tener realmente?

Tu respuesta: _____ _____

022. [Fundamentos] ¿Qué diferencia existe entre un constructor público, privado y package-private?

Tu respuesta: _____ _____

023. [Intermedio] ¿Qué son los bloques `static` e inicializadores de instancia y cuándo pueden ser problemáticos?

Tu respuesta: _____ _____

024. [Intermedio] ¿Qué diferencia hay entre una variable local, un atributo de instancia y un atributo `static`?

Tu respuesta: _____ _____

025. [Intermedio] ¿Qué significa que un campo sea `final`? ¿Implica siempre que el objeto sea inmutable?

Tu respuesta: _____ _____

026. [Intermedio] ¿Qué es una clase inmutable? Diseña una clase inmutable y explica sus invariantes.

Tu respuesta: _____ _____

027. [Intermedio] ¿Qué son los records y qué problema resuelven frente a los POJO tradicionales?

Tu respuesta: _____ _____

028. [Intermedio] ¿Cuándo no usarías un `record` para una clase de backend?

Tu respuesta: _____ _____

029. [Intermedio] ¿Qué son las clases anidadas y cuándo pueden mejorar el diseño?

Tu respuesta: _____ _____

030. [Intermedio] ¿Qué son las sealed classes y qué beneficio aportan al modelar jerarquías cerradas?

Tu respuesta: _____ _____

031. [Intermedio] ¿Qué es el `enum` y por qué suele ser mejor que constantes enteras o strings para estados?

Tu respuesta: _____ _____

032. [Intermedio] ¿Cómo modelarías un estado de una orden para impedir combinaciones inválidas?

Tu respuesta: _____ _____

033. [Intermedio] ¿Qué diferencias hay entre `null`, un valor vacío y la ausencia representada con `Optional`?

Tu respuesta: _____ _____

034. [Intermedio] ¿Por qué `Optional` no suele ser buena idea como campo de entidad JPA?

Tu respuesta: _____ _____

035. [Intermedio] ¿Cómo funciona el autoboxing/unboxing? ¿Qué bugs puede introducir con `null`?

Tu respuesta: _____ _____

036. [Intermedio] ¿Qué es un `NullPointerException` y cómo diseñarías el código para reducir su aparición?

Tu respuesta: _____ _____

037. [Intermedio] ¿Qué significa programación defensiva en Java?

Tu respuesta: _____ _____

038. [Intermedio] ¿Qué hace `var` y qué no hace? ¿Cambió el tipado de Java?

Tu respuesta: _____ _____

039. [Intermedio] ¿Qué ventajas y desventajas tiene inferir tipos con `var` en código de producción?

Tu respuesta: _____ _____

040. [Intermedio] ¿Qué son los paquetes y qué problema de organización y visibilidad ayudan a resolver?

Tu respuesta: _____ _____

041. [Intermedio] ¿Qué diferencia hay entre import explícito, wildcard y clases del mismo paquete?

Tu respuesta: _____ _____

042. [Intermedio] ¿Cómo funciona el orden de resolución de métodos cuando hay herencia y overloads?

Tu respuesta: _____ _____

043. [Intermedio] ¿Qué es covarianza de retorno? Da un ejemplo de overriding.

Tu respuesta: _____ _____

044. [Avanzado] ¿Qué significa que una excepción sea checked o unchecked?

Tu respuesta: _____ _____

045. [Avanzado] ¿Cómo decidirías entre lanzar una excepción, devolver `Optional` o devolver un resultado con estado?

Tu respuesta: _____ _____

046. [Avanzado] ¿Por qué no conviene capturar `Exception` indiscriminadamente?

Tu respuesta: _____ _____

047. [Avanzado] ¿Cómo diseñarías una jerarquía de excepciones para una API bancaria?

Tu respuesta: _____ _____

048. [Avanzado] ¿Qué diferencia hay entre `throw` y `throws`?

Tu respuesta: _____ _____

049. [Avanzado] ¿Qué recursos deberían ir en `try-with-resources` y por qué?

Tu respuesta: _____ _____

050. [Avanzado] ¿Qué es `AutoCloseable` y cómo crearías tu propio recurso administrable?

Tu respuesta: _____ _____

051. [Avanzado] ¿Qué sucede si una excepción ocurre en el bloque `try` y otra en `close()`?

Tu respuesta: _____ _____

052. [Avanzado] ¿Cómo funciona `finally` y en qué casos puede no ejecutarse como esperas?

Tu respuesta: _____ _____

053. [Avanzado] ¿Qué diferencia hay entre una checked exception de infraestructura y una excepción de dominio?

Tu respuesta: _____ _____

054. [Avanzado] ¿Qué hace `assert` y por qué no debería sustituir validaciones de entrada?

Tu respuesta: _____ _____

055. [Avanzado] ¿Cómo funciona `switch` moderno y cuándo usarías una expresión `switch`?

Tu respuesta: _____ _____

056. [Avanzado] ¿Qué problema resuelven los textos `String.format`, `Formatter` y text blocks frente a concatenaciones?

Tu respuesta: _____ _____

057. [Avanzado] ¿Qué son los annotations de Java y cuál es la diferencia entre una anotación y una interfaz?

Tu respuesta: _____ _____

058. [Avanzado] ¿Qué es reflection y por qué frameworks como Spring la utilizan?

Tu respuesta: _____ _____

059. [Avanzado] ¿Qué costes y riesgos introduce reflection?

Tu respuesta: _____ _____

060. [Avanzado] ¿Qué es un lambda en Java y qué representa conceptualmente?

Tu respuesta: _____ _____

061. [Avanzado] ¿Qué significa que una operación de Stream sea lazy?

Tu respuesta: _____ _____

062. [Avanzado] ¿Cuándo preferirías un `for` tradicional sobre un Stream?

Tu respuesta: _____ _____

063. [Avanzado] ¿Qué errores de diseño aparecen al usar Streams excesivamente?

Tu respuesta: _____ _____

064. [Avanzado] ¿Qué diferencia hay entre `map`, `flatMap` y `filter`? Explícala con datos anidados.

Tu respuesta: _____ _____

065. [Avanzado] ¿Qué significa que una función sea side-effect free? ¿Por qué importa en Streams?

Tu respuesta: _____ _____

02. Collections, Generics y programación funcional

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

066. [Fundamentos] ¿Cuándo elegirías `ArrayList` frente a `LinkedList` y por qué?

Tu respuesta: _____ _____

067. [Fundamentos] ¿Qué complejidades aproximadas tienen acceso, inserción y eliminación en `ArrayList`?

Tu respuesta: _____ _____

068. [Fundamentos] ¿Cómo crece internamente un `ArrayList` y por qué importa cuando procesas grandes volúmenes?

Tu respuesta: _____ _____

069. [Fundamentos] ¿Cuándo usarías `HashSet` y qué garantía de orden ofrece?

Tu respuesta: _____ _____

070. [Fundamentos] ¿Cómo funciona conceptualmente un `HashMap`?

Tu respuesta: _____ _____

071. [Fundamentos] ¿Qué sucede ante colisiones en un `HashMap`?

Tu respuesta: _____ _____

072. [Fundamentos] ¿Por qué mutar una clave después de insertarla en un `HashMap` puede romper el acceso?

Tu respuesta: _____ _____

073. [Fundamentos] ¿Qué diferencias hay entre `HashMap`, `LinkedHashMap` y `TreeMap`?

Tu respuesta: _____ _____

074. [Fundamentos] ¿Qué diferencias hay entre `HashSet`, `LinkedHashSet` y `TreeSet`?

Tu respuesta: _____ _____

075. [Fundamentos] ¿Qué significa que una colección sea ordenada, sorteada o simplemente estable?

Tu respuesta: _____ _____

076. [Fundamentos] ¿Qué son las colas y las deques? ¿Cuándo usarías `ArrayDeque`?

Tu respuesta: _____ _____

077. [Fundamentos] ¿Por qué `Stack` suele evitarse en código moderno?

Tu respuesta: _____ _____

078. [Fundamentos] ¿Qué diferencia existe entre `PriorityQueue` y una cola FIFO tradicional?

Tu respuesta: _____ _____

079. [Fundamentos] ¿Qué es `ConcurrentHashMap` y qué problema resuelve?

Tu respuesta: _____ _____

080. [Intermedio] ¿Qué ocurre si modificas una colección mientras la recorres?

Tu respuesta: _____ _____

081. [Intermedio] ¿Qué es `Collections.unmodifiableList()` y en qué se diferencia de una colección verdaderamente inmutable?

Tu respuesta: _____ _____

082. [Intermedio] ¿Qué ofrece `List.copyOf()` frente a una lista mutable?

Tu respuesta: _____ _____

083. [Intermedio] ¿Qué significa que una colección sea covariante o invariante en Java?

Tu respuesta: _____ _____

084. [Intermedio] ¿Qué diferencia hay entre `List<Object>`, `List<?>` y `List<? extends Object>`?

Tu respuesta: _____

085. [Intermedio] ¿Cuándo usar `? extends T` y cuándo `? super T`?

Tu respuesta: _____

086. [Intermedio] ¿Explica la regla PECS con un ejemplo de API genérica.

Tu respuesta: _____

087. [Intermedio] ¿Por qué los genéricos de Java sufren type erasure?

Tu respuesta: _____

088. [Intermedio] ¿Qué restricciones produce type erasure al intentar crear ciertos tipos genéricos?

Tu respuesta: _____

089. [Intermedio] ¿Qué es un método genérico y cuándo usarías uno?

Tu respuesta: _____

090. [Intermedio] ¿Qué diferencia hay entre un tipo raw y un tipo parametrizado?

Tu respuesta: _____

091. [Intermedio] ¿Qué riesgos introduce un cast unchecked y cómo evitarías su propagación?

Tu respuesta: _____

092. [Intermedio] ¿Qué ventajas ofrece `Optional.map()` frente a `if (x != null)` en ciertos casos?

Tu respuesta: _____

093. [Avanzado] ¿Qué errores de diseño son frecuentes al encadenar `Optional`?

Tu respuesta: _____

094. [Avanzado] ¿Cómo funciona `Collectors.groupingBy()` y qué problema de backend podría resolver?

Tu respuesta: _____

095. [Avanzado] ¿Qué diferencia hay entre `Collectors.toList()`, `toUnmodifiableList()` y `toSet()`?

Tu respuesta: _____

096. [Avanzado] ¿Cómo recolectarías un Stream en un `Map` y qué pasa si existen claves duplicadas?

Tu respuesta: _____

097. [Avanzado] ¿Qué significa que una operación de reducción requiera una identidad?

Tu respuesta: _____

098. [Avanzado] ¿Cuándo un Stream paralelo puede empeorar el rendimiento?

Tu respuesta: _____

099. [Avanzado] ¿Qué tipos de tareas son candidatas para procesamiento paralelo y cuáles no?

Tu respuesta: _____

100. [Avanzado] ¿Qué es `peek()` y por qué abusar de él puede ser una mala práctica?

Tu respuesta: _____

101. [Avanzado] ¿Cómo ordenarías por múltiples criterios y con nulls explícitos?

Tu respuesta: _____

102. [Avanzado] ¿Qué estructura elegirías para cachear valores por clave con expiración externa?

Tu respuesta: _____

103. [Avanzado] ¿Cuándo una colección mutable compartida se convierte en un problema de concurrencia?

Tu respuesta: _____ _____

104. **[Avanzado]** ¿Cómo decidirías entre devolver una lista vacía o `null` desde un método?

Tu respuesta: _____ _____

105. **[Avanzado]** ¿Qué invariantes debería preservar una colección de objetos de dominio?

Tu respuesta: _____ _____

03. JVM, memoria, concurrencia y rendimiento

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

106. [Fundamentos] ¿Qué diferencia hay entre JDK, JRE y JVM?

Tu respuesta: _____ _____

107. [Fundamentos] ¿Qué problema resuelve la JVM respecto de la portabilidad de Java?

Tu respuesta: _____ _____

108. [Fundamentos] ¿Qué parte de Java define el lenguaje y qué parte define la máquina virtual?

Tu respuesta: _____ _____

109. [Fundamentos] ¿Qué es la JLS y qué relación tiene con la especificación de la JVM?

Tu respuesta: _____ _____

110. [Fundamentos] ¿Qué es bytecode y por qué Java lo genera antes de ejecutar el programa?

Tu respuesta: _____ _____

111. [Fundamentos] ¿Qué contiene un archivo `.class` a alto nivel?

Tu respuesta: _____ _____

112. [Fundamentos] ¿Qué hace el class loader y por qué existen distintos niveles de carga de clases?

Tu respuesta: _____ _____

113. [Fundamentos] ¿Qué diferencia hay entre cargar, enlazar e inicializar una clase?

Tu respuesta: _____ _____

114. [Fundamentos] ¿Qué ocurre cuando una clase se inicializa por primera vez?

Tu respuesta: _____ _____

115. [Fundamentos] ¿Qué función cumple la `main` class y qué hace `SpringApplication.run()` a alto nivel?

Tu respuesta: _____ _____

116. [Fundamentos] ¿Qué es el heap y qué tipo de objetos viven allí?

Tu respuesta: _____ _____

117. [Fundamentos] ¿Qué es el stack de un thread y qué información suele contener un frame?

Tu respuesta: _____ _____

118. [Fundamentos] ¿Qué es Metaspace y qué problema reemplazó frente a PermGen?

Tu respuesta: _____ _____

119. [Fundamentos] ¿Qué diferencia existe entre memoria de proceso y memoria administrada por el heap?

Tu respuesta: _____ _____

120. [Fundamentos] ¿Cómo funciona el garbage collector de forma conceptual?

Tu respuesta: _____ _____

121. [Fundamentos] ¿Qué significa que un objeto sea elegible para GC?

Tu respuesta: _____ _____

122. [Fundamentos] ¿Por qué un memory leak puede existir incluso con garbage collection?

Tu respuesta: _____ _____

123. [Fundamentos] ¿Qué síntomas diferenciarían un leak de memoria de una alta presión normal del GC?

Tu respuesta: _____ _____

124. [Fundamentos] ¿Qué es stop-the-world y por qué importa en un servidor?

Tu respuesta: _____ _____

125. [Intermedio] ¿Qué información mirarías primero al investigar pausas de GC?

Tu respuesta: _____ _____

126. [Intermedio] ¿Qué son G1, ZGC y otros recolectores y qué trade-offs persiguen?

Tu respuesta: _____ _____

127. [Intermedio] ¿Qué es JIT y cómo mejora el rendimiento de una aplicación Java?

Tu respuesta: _____ _____

128. [Intermedio] ¿Qué significa que HotSpot caliente ciertos métodos?

Tu respuesta: _____ _____

129. [Intermedio] ¿Qué optimizaciones puede realizar un JIT y qué límites tiene?

Tu respuesta: _____ _____

130. [Intermedio] ¿Qué es escape analysis y qué tipo de optimización puede habilitar?

Tu respuesta: _____ _____

131. [Intermedio] ¿Qué es inlining y por qué una función pequeña no necesariamente cuesta una llamada?

Tu respuesta: _____ _____

132. [Intermedio] ¿Por qué un microbenchmark Java mal diseñado puede mentirte?

Tu respuesta: _____ _____

133. [Intermedio] ¿Qué papel cumple JMH en benchmarking?

Tu respuesta: _____ _____

134. [Intermedio] ¿Qué es false sharing y en qué situaciones puede aparecer?

Tu respuesta: _____ _____

135. [Intermedio] ¿Qué diferencia hay entre latencia, throughput y utilización?

Tu respuesta: _____ _____

136. [Intermedio] ¿Qué es un thread y qué relación tiene con una tarea (`Runnable`/`Callable`)?

Tu respuesta: _____ _____

137. [Intermedio] ¿Qué diferencia hay entre `Thread`, `ExecutorService` y `CompletableFuture`?

Tu respuesta: _____ _____

138. [Intermedio] ¿Qué es un thread pool y por qué suele ser mejor que crear threads ilimitadamente?

Tu respuesta: _____ _____

139. [Intermedio] ¿Cómo dimensionarías un pool para tareas CPU-bound frente a I/O-bound?

Tu respuesta: _____ _____

140. [Intermedio] ¿Qué problema resuelve `Future` y qué limita frente a `CompletableFuture`?

Tu respuesta: _____ _____

141. [Intermedio] ¿Cómo encadenarías tareas asíncronas y manejo de errores con `CompletableFuture`?

Tu respuesta: _____ _____

142. [Intermedio] ¿Qué significa thread safety?

Tu respuesta: _____ _____

143. [Avanzado] ¿Qué es una race condition? Construye un ejemplo conceptual.

Tu respuesta: _____ _____

144. [Avanzado] ¿Qué es una condición de carrera de tipo check-then-act?

Tu respuesta: _____ _____

145. [Avanzado] ¿Qué diferencia hay entre atomicidad, visibilidad y ordenamiento de memoria?

Tu respuesta: _____ _____

146. [Avanzado] ¿Qué hace `synchronized` y qué garantías aporta?

Tu respuesta: _____ _____

147. [Avanzado] ¿Qué diferencia hay entre sincronizar un bloque y sincronizar un método?

Tu respuesta: _____ _____

148. [Avanzado] ¿Qué es un monitor en Java?

Tu respuesta: _____ _____

149. [Avanzado] ¿Qué problema resuelve `volatile` y cuál no resuelve?

Tu respuesta: _____ _____

150. [Avanzado] ¿Por qué `volatile` no convierte `counter++` en una operación atómica?

Tu respuesta: _____ _____

151. [Avanzado] ¿Qué son `AtomicInteger` y clases atómicas y cuándo usarlas?

Tu respuesta: _____ _____

152. [Avanzado] ¿Qué es `Lock` y cuándo preferirías `ReentrantLock` a `synchronized`?

Tu respuesta: _____ _____

153. [Avanzado] ¿Qué es deadlock? Enumera las condiciones necesarias para que ocurra.

Tu respuesta: _____ _____

154. [Avanzado] ¿Qué es livelock y cómo se diferencia de deadlock?

Tu respuesta: _____ _____

155. [Avanzado] ¿Qué es starvation?

Tu respuesta: _____ _____

156. [Avanzado] ¿Qué información aporta un thread dump?

Tu respuesta: _____ _____

157. [Avanzado] ¿Qué información aporta un heap dump?

Tu respuesta: _____ _____

158. [Avanzado] ¿Qué métricas de JVM vigilarías en un backend de alto tráfico?

Tu respuesta: _____ _____

159. [Avanzado] ¿Qué diferencia hay entre optimizar CPU, memoria, I/O y asignaciones?

Tu respuesta: _____ _____

160. [Avanzado] ¿Por qué medir antes de optimizar es especialmente importante en Java?

Tu respuesta: _____ _____

04. Maven, dependencias y build

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

161. [Fundamentos] ¿Qué problema resuelve Maven en un proyecto Java?

Tu respuesta: _____ _____

162. [Fundamentos] ¿Qué diferencia hay entre código fuente, artefacto y proyecto Maven?

Tu respuesta: _____ _____

163. [Fundamentos] ¿Qué contiene un `pom.xml` y qué decisiones centraliza?

Tu respuesta: _____ _____

164. [Fundamentos] ¿Qué significa `groupId`, `artifactId` y `version`?

Tu respuesta: _____ _____

165. [Fundamentos] ¿Qué es una dependencia directa y qué es una dependencia transitiva?

Tu respuesta: _____ _____

166. [Fundamentos] ¿Cómo resuelve Maven una dependencia transitiva?

Tu respuesta: _____ _____

167. [Fundamentos] ¿Qué pasa si dos dependencias traen versiones incompatibles de la misma librería?

Tu respuesta: _____ _____

168. [Fundamentos] ¿Qué significa dependency mediation en Maven?

Tu respuesta: _____ _____

169. [Fundamentos] ¿Qué es `dependencyManagement` y por qué no añade por sí mismo una dependencia?

Tu respuesta: _____ _____

170. [Fundamentos] ¿Qué diferencia hay entre `dependencies` y `dependencyManagement`?

Tu respuesta: _____ _____

171. [Intermedio] ¿Qué problema resuelve un Maven Wrapper?

Tu respuesta: _____ _____

172. [Intermedio] ¿Por qué `mvnw` puede ser importante en CI y equipos?

Tu respuesta: _____ _____

173. [Intermedio] ¿Qué diferencia hay entre instalar Maven globalmente y usar el wrapper?

Tu respuesta: _____ _____

174. [Intermedio] ¿Qué es el lifecycle de Maven?

Tu respuesta: _____ _____

175. [Intermedio] ¿Cuáles son los lifecycle principales (`clean`, `default`, `site`) y qué propósito tiene cada uno?

Tu respuesta: _____ _____

176. [Intermedio] ¿Qué diferencia hay entre `compile`, `test`, `package`, `verify` e `install`?

Tu respuesta: _____ _____

177. [Intermedio] ¿Qué significa que una fase active fases anteriores?

Tu respuesta: _____ _____

178. [Intermedio] ¿Qué hace `mvn clean package` realmente?

Tu respuesta: _____ _____

179. [Intermedio] ¿Qué es un plugin de Maven?

Tu respuesta: _____ _____

180. [Intermedio] ¿Qué diferencia hay entre dependencia y plugin?

Tu respuesta: _____ _____

181. [Avanzado] ¿Qué es `maven-compiler-plugin` y por qué importa la versión de Java que configura?

Tu respuesta: _____ _____

182. [Avanzado] ¿Cómo definirías Java 21 correctamente en un proyecto Maven moderno?

Tu respuesta: _____ _____

183. [Avanzado] ¿Qué son los scopes `compile`, `provided`, `runtime`, `test` y `optional`?

Tu respuesta: _____ _____

184. [Avanzado] ¿Cuándo usarías `runtime` en una aplicación Spring Boot?

Tu respuesta: _____ _____

185. [Avanzado] ¿Qué diferencia hay entre `spring-boot-starter` y una librería individual?

Tu respuesta: _____ _____

186. [Avanzado] ¿Qué hace `spring-boot-maven-plugin`?

Tu respuesta: _____ _____

187. [Avanzado] ¿Qué significa que un JAR sea executable/fat/uber JAR en Spring Boot?

Tu respuesta: _____ _____

188. [Avanzado] ¿Qué es un repository local de Maven?

Tu respuesta: _____ _____

189. [Avanzado] ¿Qué son Maven Central y los repositories privados?

Tu respuesta: _____ _____

190. [Avanzado] ¿Qué es un BOM y cuándo usarías uno?

Tu respuesta: _____ _____

05. Spring Core: IoC, DI, Beans y AOP

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

191. [Fundamentos] ¿Qué problema intenta resolver Spring frente a crear todas las dependencias manualmente con `new`?

Tu respuesta: _____ _____

192. [Fundamentos] ¿Qué es IoC (Inversion of Control)? Explícalo sin usar definiciones memorizadas.

Tu respuesta: _____ _____

193. [Fundamentos] ¿Qué es Dependency Injection y cómo se relaciona con IoC?

Tu respuesta: _____ _____

194. [Fundamentos] ¿Qué diferencias hay entre constructor injection, setter injection y field injection?

Tu respuesta: _____ _____

195. [Fundamentos] ¿Por qué constructor injection suele ser preferible en servicios?

Tu respuesta: _____ _____

196. [Fundamentos] ¿Qué ventaja de testing aporta constructor injection?

Tu respuesta: _____ _____

197. [Fundamentos] ¿Qué es un Spring Bean?

Tu respuesta: _____ _____

198. [Fundamentos] ¿Quién crea un Bean y quién controla su ciclo de vida?

Tu respuesta: _____ _____

199. [Fundamentos] ¿Qué es `ApplicationContext`?

Tu respuesta: _____ _____

200. [Fundamentos] ¿Qué relación existe entre `ApplicationContext` y el contenedor de IoC?

Tu respuesta: _____ _____

201. [Fundamentos] ¿Qué hace `@Component`?

Tu respuesta: _____ _____

202. [Fundamentos] ¿Qué diferencia hay entre `@Component`, `@Service`, `@Repository` y `@Controller`?

Tu respuesta: _____ _____

203. [Fundamentos] ¿Son `@Service` y `@Component` diferentes en el mecanismo básico de registro?

Tu respuesta: _____ _____

204. [Fundamentos] ¿Cómo decide Spring qué implementación inyectar cuando hay una interfaz?

Tu respuesta: _____ _____

205. [Fundamentos] ¿Qué problema resuelve `@Primary`?

Tu respuesta: _____ _____

206. [Intermedio] ¿Qué problema resuelve `@Qualifier`?

Tu respuesta: _____ _____

207. [Intermedio] ¿Qué ocurre si existen dos Beans candidatos y ninguno está marcado como principal?

Tu respuesta: _____ _____

208. [Intermedio] ¿Qué es un Bean singleton en Spring? ¿Significa lo mismo que singleton del GoF?

Tu respuesta: _____ _____

209. [Intermedio] ¿Qué otros scopes de Bean existen y cuándo usarías alguno?

Tu respuesta: _____ _____

210. [Intermedio] ¿Cuándo se ejecuta un constructor de Bean singleton?

Tu respuesta: _____ _____

211. [Intermedio] ¿Qué hacen `@PostConstruct` y `@PreDestroy`?

Tu respuesta: _____ _____

212. [Intermedio] ¿Qué ocurre si un Bean depende de otro Bean?

Tu respuesta: _____ _____

213. [Intermedio] ¿Cómo resolverías una dependencia circular entre dos servicios?

Tu respuesta: _____ _____

214. [Intermedio] ¿Por qué una dependencia circular puede indicar un problema de diseño?

Tu respuesta: _____ _____

215. [Intermedio] ¿Qué hace un método `@Bean` dentro de una clase `@Configuration`?

Tu respuesta: _____ _____

216. [Intermedio] ¿Por qué `@Configuration` tiene un tratamiento especial frente a una clase normal?

Tu respuesta: _____ _____

217. [Intermedio] ¿Qué significa component scanning?

Tu respuesta: _____ _____

218. [Intermedio] ¿Cómo decide Spring qué paquetes escanear?

Tu respuesta: _____ _____

219. [Intermedio] ¿Qué problemas aparecen si mueves la clase principal a un paquete inadecuado?

Tu respuesta: _____ _____

220. [Intermedio] ¿Qué son los profiles de Spring y para qué sirven?

Tu respuesta: _____ _____

221. [Avanzado] ¿Cómo crearías diferentes implementaciones de un Bean para test, desarrollo y producción?

Tu respuesta: _____ _____

222. [Avanzado] ¿Qué diferencia hay entre configuración explícita con `@Bean` y autodetección con `@Component`?

Tu respuesta: _____ _____

223. [Avanzado] ¿Cuándo es preferible registrar un Bean explícitamente?

Tu respuesta: _____ _____

224. [Avanzado] ¿Qué son las properties y cómo llega una propiedad desde `application.yml` a un Bean?

Tu respuesta: _____ _____

225. [Avanzado] ¿Qué hace `@Value` y qué limitaciones tiene?

Tu respuesta: _____ _____

226. [Avanzado] ¿Por qué `@ConfigurationProperties` suele ser mejor para grupos de configuración?

Tu respuesta: _____ _____

227. [Avanzado] ¿Qué significa externalizar configuración?

Tu respuesta: _____ _____

228. [Avanzado] ¿Qué es Spring Environment?

Tu respuesta: _____ _____

229. [Avanzado] ¿Qué es un `BeanPostProcessor` a alto nivel?

Tu respuesta: _____ _____

230. [Avanzado] ¿Por qué Spring usa proxies dinámicos o generados?

Tu respuesta: _____ _____

231. [Avanzado] ¿Qué es self-invocation y por qué puede romper expectativas con proxies de Spring?

Tu respuesta: _____ _____

232. [Avanzado] ¿Qué diferencia hay entre un proxy JDK y un proxy basado en subclases?

Tu respuesta: _____ _____

233. [Avanzado] ¿Qué problemas de diseño aparecen al depender directamente del contenedor Spring en las clases de dominio?

Tu respuesta: _____ _____

234. [Avanzado] ¿Qué significa que una clase sea POJO y por qué sigue siendo útil en Spring?

Tu respuesta: _____ _____

235. [Avanzado] ¿Cómo explicarías el ciclo de vida de un Bean desde registro hasta destrucción?

Tu respuesta: _____ _____

06. Spring Boot, Web, MVC y configuración

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

236. [Fundamentos] ¿Qué añade Spring Boot sobre Spring Framework?

Tu respuesta: _____ _____

237. [Fundamentos] ¿Qué hace realmente `@SpringBootApplication`? Descompón sus responsabilidades.

Tu respuesta: _____ _____

238. [Fundamentos] ¿Qué hace `SpringApplication.run()` a alto nivel?

Tu respuesta: _____ _____

239. [Fundamentos] ¿Qué es auto-configuration?

Tu respuesta: _____ _____

240. [Fundamentos] ¿Cómo decide Spring Boot qué auto-configuraciones aplicar?

Tu respuesta: _____ _____

241. [Fundamentos] ¿Qué papel juegan las condiciones de auto-configuration?

Tu respuesta: _____ _____

242. [Fundamentos] ¿Qué significa “convention over configuration” en Spring Boot?

Tu respuesta: _____ _____

243. [Fundamentos] ¿Qué es un starter de Spring Boot?

Tu respuesta: _____ _____

244. [Fundamentos] ¿Cómo crearías una API REST mínima con Spring Boot?

Tu respuesta: _____ _____

245. [Fundamentos] ¿Qué es `DispatcherServlet` y qué papel tiene en Spring MVC?

Tu respuesta: _____ _____

246. [Fundamentos] ¿Cómo viaja una petición HTTP desde el servidor hasta un método `@RestController`?

Tu respuesta: _____ _____

247. [Fundamentos] ¿Qué diferencia hay entre `@Controller` y `@RestController`?

Tu respuesta: _____ _____

248. [Fundamentos] ¿Qué hace `@RequestMapping`?

Tu respuesta: _____ _____

249. [Fundamentos] ¿Qué diferencias hay entre `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping` y `@DeleteMapping`?

Tu respuesta: _____ _____

250. [Fundamentos] ¿Cómo extraerías un path variable y cuándo usarías uno?

Tu respuesta: _____ _____

251. [Fundamentos] ¿Cómo leerías query parameters y qué defaults puedes definir?

Tu respuesta: _____ _____

252. [Fundamentos] ¿Qué hace `@RequestBody` y cómo se convierte JSON a un objeto Java?

Tu respuesta: _____ _____

253. [Intermedio] ¿Qué papel cumple Jackson en una API Spring Boot típica?

Tu respuesta: _____ _____

254. [Intermedio] ¿Qué ocurre si el JSON entrante no puede deserializarse?

Tu respuesta: _____ _____

255. [Intermedio] ¿Qué es content negotiation?

Tu respuesta: _____ _____

256. [Intermedio] ¿Qué diferencia hay entre `@ResponseBody` y `ResponseEntity`?

Tu respuesta: _____ _____

257. [Intermedio] ¿Cuándo devolverías `ResponseEntity`?

Tu respuesta: _____ _____

258. [Intermedio] ¿Cómo modelarías correctamente un `201 Created`?

Tu respuesta: _____ _____

259. [Intermedio] ¿Qué semántica tienen `200`, `201`, `204`, `400`, `401`, `403`, `404`, `409` y `500`?

Tu respuesta: _____ _____

260. [Intermedio] ¿Cómo centralizarías el manejo de excepciones con `@ControllerAdvice`?

Tu respuesta: _____ _____

261. [Intermedio] ¿Qué es `@ExceptionHandler` y cómo se relaciona con `@ControllerAdvice`?

Tu respuesta: _____ _____

262. [Intermedio] ¿Cómo diseñarías un formato consistente de error para toda la API?

Tu respuesta: _____ _____

263. [Intermedio] ¿Qué diferencia hay entre DTO de request, DTO de response y entidad?

Tu respuesta: _____ _____

264. [Intermedio] ¿Por qué no conviene exponer directamente una entidad JPA como response?

Tu respuesta: _____ _____

265. [Intermedio] ¿Qué problemas aparecen con lazy loading al serializar entidades?

Tu respuesta: _____ _____

266. [Intermedio] ¿Cómo validarías un request DTO?

Tu respuesta: _____ _____

267. [Intermedio] ¿Qué hacen `@Valid` y `@Validated` y cuándo usarías cada uno?

Tu respuesta: _____ _____

268. [Intermedio] ¿Qué es Bean Validation y qué resuelve?

Tu respuesta: _____ _____

269. [Avanzado] ¿Cómo crearías una validación personalizada?

Tu respuesta: _____ _____

270. [Avanzado] ¿Dónde debería ocurrir una validación de formato y dónde una regla de negocio?

Tu respuesta: _____ _____

271. [Avanzado] ¿Qué es un interceptor y cuándo usarlo?

Tu respuesta: _____ _____

272. [Avanzado] ¿Qué diferencia hay entre Filter, Interceptor y AOP en Spring?

Tu respuesta: _____ _____

273. [Avanzado] ¿Qué hace un `OncePerRequestFilter`?

Tu respuesta: _____ _____

274. [Avanzado] ¿Cómo leerías un header HTTP en un controller?

Tu respuesta: _____ _____

275. [Avanzado] ¿Qué es CORS y qué problema resuelve?

Tu respuesta: _____ _____

276. [Avanzado] ¿Qué es CSRF y cuándo importa en una API?

Tu respuesta: _____ _____

277. [Avanzado] ¿Qué diferencia existe entre una API stateful y stateless?

Tu respuesta: _____ _____

278. [Avanzado] ¿Cómo gestionarías el versionado de una API?

Tu respuesta: _____ _____

279. [Avanzado] ¿Dónde pondrías el versionado: URL, header o content type? ¿Qué trade-offs hay?

Tu respuesta: _____ _____

280. [Avanzado] ¿Cómo parametrizarías una API para paginación, orden y filtros?

Tu respuesta: _____ _____

281. [Avanzado] ¿Cómo organizarías paquetes de un proyecto Spring Boot mediano?

Tu respuesta: _____ _____

282. [Avanzado] ¿Qué diferencias hay entre estructura por capas y estructura orientada a features?

Tu respuesta: _____ _____

283. [Avanzado] ¿Cómo leería Spring una configuración desde `application.yml`?

Tu respuesta: _____ _____

284. [Avanzado] ¿Qué orden de precedencia existe entre distintas fuentes de configuración de Spring Boot?

Tu respuesta: _____ _____

285. [Avanzado] ¿Cómo separarías configuración sensible de configuración normal?

Tu respuesta: _____ _____

07. JPA, Hibernate y persistencia

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

286. [Fundamentos] ¿Qué problema resuelve JPA?

Tu respuesta: _____ _____

287. [Fundamentos] ¿JPA es una implementación? ¿Cuál es la diferencia entre especificación e implementación?

Tu respuesta: _____ _____

288. [Fundamentos] ¿Qué es Hibernate y qué relación tiene con JPA?

Tu respuesta: _____ _____

289. [Fundamentos] ¿Qué significa ORM?

Tu respuesta: _____ _____

290. [Fundamentos] ¿Qué ventajas y costes introduce un ORM frente a SQL escrito manualmente?

Tu respuesta: _____ _____

291. [Fundamentos] ¿Qué convierte una clase Java en una entidad JPA?

Tu respuesta: _____ _____

292. [Fundamentos] ¿Qué papel cumple `@Entity`?

Tu respuesta: _____ _____

293. [Fundamentos] ¿Por qué una entidad JPA necesita un constructor sin argumentos en muchos casos?

Tu respuesta: _____ _____

294. [Fundamentos] ¿Qué es el identificador de una entidad y cómo se genera?

Tu respuesta: _____ _____

295. [Fundamentos] ¿Qué estrategias de generación de ID existen y qué trade-offs tienen?

Tu respuesta: _____ _____

296. [Fundamentos] ¿Qué significa que una entidad esté en estado transient, managed, detached o removed?

Tu respuesta: _____ _____

297. [Fundamentos] ¿Qué es el persistence context?

Tu respuesta: _____ _____

298. [Fundamentos] ¿Qué es el primer nivel de caché de JPA?

Tu respuesta: _____ _____

299. [Fundamentos] ¿Qué significa dirty checking?

Tu respuesta: _____ _____

300. [Fundamentos] ¿Cuándo Hibernate detecta cambios en una entidad y cuándo los sincroniza?

Tu respuesta: _____ _____

301. [Fundamentos] ¿Qué hace `flush()` y qué no hace?

Tu respuesta: _____ _____

302. [Fundamentos] ¿Qué diferencia hay entre `flush`, `commit` y `save`?

Tu respuesta: _____ _____

303. [Fundamentos] ¿Qué ocurre cuando llamas `repository.save(entity)` dentro de una transacción?

Tu respuesta: _____ _____

304. [Fundamentos] ¿Qué problemas puede producir guardar entidades innecesariamente?

Tu respuesta: _____ _____

305. [Intermedio] ¿Qué diferencia hay entre `findById` y una consulta que devuelve una lista?

Tu respuesta: _____ _____

306. [Intermedio] ¿Qué es Spring Data JPA?

Tu respuesta: _____ _____

307. [Intermedio] ¿Qué problema resuelve un `JpaRepository`?

Tu respuesta: _____ _____

308. [Intermedio] ¿Qué métodos básicos hereda `JpaRepository`?

Tu respuesta: _____ _____

309. [Intermedio] ¿Cómo funciona el query derivation de Spring Data JPA?

Tu respuesta: _____ _____

310. [Intermedio] ¿Cuándo prefieres una query derivada frente a `@Query`?

Tu respuesta: _____ _____

311. [Intermedio] ¿Qué es JPQL y en qué se diferencia de SQL?

Tu respuesta: _____ _____

312. [Intermedio] ¿Qué es una native query y cuándo podría ser necesaria?

Tu respuesta: _____ _____

313. [Intermedio] ¿Qué es `Specification` y qué problema de filtros dinámicos puede resolver?

Tu respuesta: _____ _____

314. [Intermedio] ¿Qué son los projections de Spring Data JPA?

Tu respuesta: _____ _____

315. [Intermedio] ¿Cuándo usarías una projection en vez de cargar una entidad completa?

Tu respuesta: _____ _____

316. [Intermedio] ¿Qué es lazy loading?

Tu respuesta: _____ _____

317. [Intermedio] ¿Qué es eager loading?

Tu respuesta: _____ _____

318. [Intermedio] ¿Por qué `EAGER` no es automáticamente mejor que `LAZY`?

Tu respuesta: _____ _____

319. [Intermedio] ¿Qué es el problema N+1?

Tu respuesta: _____ _____

320. [Intermedio] ¿Cómo detectarías un N+1 en una aplicación Spring Boot?

Tu respuesta: _____ _____

321. [Intermedio] ¿Cómo resolverías un N+1 con `join fetch`?

Tu respuesta: _____ _____

322. [Intermedio] ¿Qué limitaciones tiene `join fetch` con paginación?

Tu respuesta: _____ _____

323. [Avanzado] ¿Qué es `@EntityGraph` y cuándo resulta útil?

Tu respuesta: _____ _____

324. [Avanzado] ¿Qué diferencia hay entre `@ManyToOne`, `@OneToMany`, `@OneToOne` y `@ManyToMany`?

Tu respuesta: _____ _____

325. [Avanzado] ¿Por qué `@ManyToMany` suele requerir un diseño cuidadoso?

Tu respuesta: _____ _____

326. [Avanzado] ¿Cuándo conviene modelar explícitamente la tabla intermedia como una entidad?

Tu respuesta: _____ _____

327. [Avanzado] ¿Qué significa owning side en una relación JPA?

Tu respuesta: _____ _____

328. [Avanzado] ¿Qué hace `mappedBy`?

Tu respuesta: _____ _____

329. [Avanzado] ¿Qué problemas aparecen si no mantienes ambos lados de una relación bidireccional en sync?

Tu respuesta: _____ _____

330. [Avanzado] ¿Qué son cascade types y cuáles existen?

Tu respuesta: _____ _____

331. [Avanzado] ¿Por qué `CascadeType.ALL` puede ser peligroso?

Tu respuesta: _____ _____

332. [Avanzado] ¿Qué hace `orphanRemoval`?

Tu respuesta: _____ _____

333. [Avanzado] ¿Qué es `@Embedded` y cuándo lo usarías?

Tu respuesta: _____ _____

334. [Avanzado] ¿Qué diferencia hay entre `@Embedded` y una entidad relacionada?

Tu respuesta: _____ _____

335. [Avanzado] ¿Cómo mapearías un value object?

Tu respuesta: _____ _____

336. [Avanzado] ¿Cómo manejarías un `OptimisticLockException`?

Tu respuesta: _____ _____

337. [Avanzado] ¿Qué significa propagation en `@Transactional`?

Tu respuesta: _____ _____

338. [Avanzado] ¿Qué niveles de isolation te interesan entender para trabajar con Spring?

Tu respuesta: _____ _____

339. [Avanzado] ¿Cuándo una transacción debería vivir en el service y no en el repository?

Tu respuesta: _____ _____

340. [Avanzado] ¿Qué pasa con una excepción checked vs unchecked dentro de `@Transactional`?

Tu respuesta: _____ _____

08. SQL, bases de datos y modelado de datos

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

341. [Fundamentos] ¿Qué diferencia hay entre una base de datos relacional y una no relacional?

Tu respuesta: _____ _____

342. [Fundamentos] ¿Qué problema resuelve una clave primaria?

Tu respuesta: _____ _____

343. [Fundamentos] ¿Qué diferencia hay entre clave primaria, candidata, única y foránea?

Tu respuesta: _____ _____

344. [Fundamentos] ¿Qué es integridad referencial?

Tu respuesta: _____ _____

345. [Fundamentos] ¿Qué significa normalizar una base de datos?

Tu respuesta: _____ _____

346. [Fundamentos] ¿Qué resuelven 1NF, 2NF y 3NF a grandes rasgos?

Tu respuesta: _____ _____

347. [Fundamentos] ¿Cuándo una desnormalización puede ser razonable?

Tu respuesta: _____ _____

348. [Fundamentos] ¿Qué diferencia hay entre `INNER JOIN`, `LEFT JOIN` y `RIGHT JOIN`?

Tu respuesta: _____ _____

349. [Fundamentos] ¿Qué hace un `GROUP BY` y qué diferencia tiene con `DISTINCT`?

Tu respuesta: _____ _____

350. [Fundamentos] ¿Qué problemas resuelve `HAVING` frente a `WHERE`?

Tu respuesta: _____ _____

351. [Intermedio] ¿Qué es una subquery correlacionada?

Tu respuesta: _____ _____

352. [Intermedio] ¿Qué es una CTE y cuándo mejora una consulta?

Tu respuesta: _____ _____

353. [Intermedio] ¿Qué es un índice y por qué no conviene indexar todas las columnas?

Tu respuesta: _____ _____

354. [Intermedio] ¿Cómo funciona conceptualmente un índice B-tree?

Tu respuesta: _____ _____

355. [Intermedio] ¿Qué es un índice compuesto y cómo elegirías el orden de sus columnas?

Tu respuesta: _____ _____

356. [Intermedio] ¿Qué es selectividad de un índice?

Tu respuesta: _____ _____

357. [Intermedio] ¿Cómo verificarías por qué una consulta es lenta?

Tu respuesta: _____ _____

358. [Intermedio] ¿Qué información aporta un `EXPLAIN` o `EXPLAIN ANALYZE`?

Tu respuesta: _____ _____

359. [Intermedio] ¿Qué es una secuencial scan y cuándo puede ser correcto?

Tu respuesta: _____ _____

360. [Intermedio] ¿Qué diferencia hay entre paginación por offset y keyset pagination?

Tu respuesta: _____ _____

361. [Avanzado] ¿Por qué offset pagination puede empeorar mucho con páginas profundas?

Tu respuesta: _____ _____

362. [Avanzado] ¿Qué es una transacción en SQL?

Tu respuesta: _____ _____

363. [Avanzado] ¿Qué significan ACID?

Tu respuesta: _____ _____

364. [Avanzado] ¿Qué significa atomicidad en una transferencia bancaria?

Tu respuesta: _____ _____

365. [Avanzado] ¿Qué anomalías de concurrencia existen entre transacciones?

Tu respuesta: _____ _____

366. [Avanzado] ¿Qué es connection pooling y por qué importa en Spring Boot?

Tu respuesta: _____ _____

367. [Avanzado] ¿Qué es HikariCP y qué papel cumple en una aplicación Spring?

Tu respuesta: _____ _____

368. [Avanzado] ¿Qué significa que una conexión se agote en un backend?

Tu respuesta: _____ _____

369. [Avanzado] ¿Qué ventajas y desventajas tiene usar PostgreSQL frente a H2 en tests?

Tu respuesta: _____ _____

370. [Avanzado] ¿Cuándo un test con H2 puede ocultar un bug que sí aparece en PostgreSQL?

Tu respuesta: _____ _____

09. REST, HTTP, seguridad y diseño de APIs

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

371. [Fundamentos] ¿Qué diferencia hay entre HTTP y HTTPS?

Tu respuesta: _____ _____

372. [Fundamentos] ¿Qué hace TLS y qué protege exactamente?

Tu respuesta: _____ _____

373. [Fundamentos] ¿Cuáles son los componentes principales de una petición HTTP?

Tu respuesta: _____ _____

374. [Fundamentos] ¿Qué diferencia hay entre path, query string, headers y body?

Tu respuesta: _____ _____

375. [Fundamentos] ¿Qué significa que un método HTTP sea safe?

Tu respuesta: _____ _____

376. [Fundamentos] ¿Qué significa que un método HTTP sea idempotent?

Tu respuesta: _____ _____

377. [Fundamentos] ¿Por qué `PUT` y `DELETE` se consideran idempotentes conceptualmente?

Tu respuesta: _____ _____

378. [Fundamentos] ¿Cuándo usarías `PATCH` en vez de `PUT`?

Tu respuesta: _____ _____

379. [Fundamentos] ¿Qué es cache-control y cómo puede afectar una API?

Tu respuesta: _____ _____

380. [Fundamentos] ¿Qué diferencia existe entre autenticación y autorización?

Tu respuesta: _____ _____

381. [Intermedio] ¿Qué significa el modelo de autorización RBAC?

Tu respuesta: _____ _____

382. [Intermedio] ¿Qué son scopes o authorities?

Tu respuesta: _____ _____

383. [Intermedio] ¿Cómo diseñarías roles y permisos para una API de fintech?

Tu respuesta: _____ _____

384. [Intermedio] ¿Qué es JWT y qué problema resuelve?

Tu respuesta: _____ _____

385. [Intermedio] ¿Qué información debería y no debería viajar en un JWT?

Tu respuesta: _____ _____

386. [Intermedio] ¿Qué significa que un JWT sea firmado pero no necesariamente cifrado?

Tu respuesta: _____ _____

387. [Intermedio] ¿Qué riesgos tiene almacenar un JWT en un lugar inseguro del cliente?

Tu respuesta: _____ _____

388. [Intermedio] ¿Qué es refresh token y por qué puede existir?

Tu respuesta: _____ _____

389. [Intermedio] ¿Qué diferencia hay entre session-based authentication y token-based authentication?

Tu respuesta: _____ _____

390. [Intermedio] ¿Qué es OAuth 2.0 a alto nivel?

Tu respuesta: _____ _____

391. [Avanzado] ¿Qué relación hay entre OAuth 2.0 y OpenID Connect?

Tu respuesta: _____ _____

392. [Avanzado] ¿Qué es CSRF y por qué el riesgo cambia con cookies y tokens?

Tu respuesta: _____ _____

393. [Avanzado] ¿Qué es CORS y qué no es CORS?

Tu respuesta: _____ _____

394. [Avanzado] ¿Cómo evitarías SQL injection en una aplicación JPA?

Tu respuesta: _____ _____

395. [Avanzado] ¿Qué es XSS y por qué también importa aunque desarrolles principalmente backend?

Tu respuesta: _____ _____

396. [Avanzado] ¿Qué es input validation y por qué debe existir aunque el frontend valide?

Tu respuesta: _____ _____

397. [Avanzado] ¿Qué información no debería aparecer en mensajes de error públicos?

Tu respuesta: _____ _____

398. [Avanzado] ¿Cómo diseñarías auditoría para operaciones sensibles?

Tu respuesta: _____ _____

399. [Avanzado] ¿Qué es idempotency key y qué diferencia tiene con un request ID?

Tu respuesta: _____ _____

400. [Avanzado] ¿Qué es correlation ID y cómo ayuda a depurar una petición distribuida?

Tu respuesta: _____ _____

10. Testing: unit, integration, web y calidad

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

401. [Fundamentos] ¿Qué diferencia hay entre unit test, integration test y end-to-end test?

Tu respuesta: _____ _____

402. [Fundamentos] ¿Qué característica debería tener un buen unit test?

Tu respuesta: _____ _____

403. [Fundamentos] ¿Qué significa FIRST en testing?

Tu respuesta: _____ _____

404. [Fundamentos] ¿Qué es una prueba determinista?

Tu respuesta: _____ _____

405. [Fundamentos] ¿Qué es test isolation y por qué importa?

Tu respuesta: _____ _____

406. [Fundamentos] ¿Qué hace JUnit 5 y qué partes principales deberías conocer?

Tu respuesta: _____ _____

407. [Fundamentos] ¿Qué hacen `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll` y `@AfterAll`?

Tu respuesta: _____ _____

408. [Fundamentos] ¿Qué diferencia hay entre `assertEquals`, `assertSame`, `assertThrows` y `assertAll`?

Tu respuesta: _____ _____

409. [Fundamentos] ¿Cómo escribirías un test que describa una regla de negocio importante?

Tu respuesta: _____ _____

410. [Intermedio] ¿Qué significa AAA: Arrange, Act, Assert?

Tu respuesta: _____ _____

411. [Intermedio] ¿Qué es Mockito y qué problema resuelve?

Tu respuesta: _____ _____

412. [Intermedio] ¿Cuándo usar un mock y cuándo un fake?

Tu respuesta: _____ _____

413. [Intermedio] ¿Qué diferencia hay entre mock, stub, spy y fake?

Tu respuesta: _____ _____

414. [Intermedio] ¿Qué problemas aparecen cuando mockeas demasiado?

Tu respuesta: _____ _____

415. [Intermedio] ¿Qué significa verificar una interacción y cuándo es una mala señal?

Tu respuesta: _____ _____

416. [Intermedio] ¿Cómo testarías un service que depende de un repository?

Tu respuesta: _____ _____

417. [Intermedio] ¿Cómo testarías una transacción real sin convertir todo en un test E2E?

Tu respuesta: _____ _____

418. [Avanzado] ¿Qué ofrece `@DataJpaTest`?

Tu respuesta: _____ _____

419. [Avanzado] ¿Qué ofrece `@WebMvcTest`?

Tu respuesta: _____ _____

420. [Avanzado] ¿Qué ofrece `@SpringBootTest`?

Tu respuesta: _____ _____

421. [Avanzado] ¿Qué es test pyramid y cómo la aplicarías en un backend?

Tu respuesta: _____ _____

422. [Avanzado] ¿Qué es code coverage y por qué 100% no equivale a buen testing?

Tu respuesta: _____ _____

423. [Avanzado] ¿Qué partes de un backend deberían tener mayor profundidad de tests?

Tu respuesta: _____ _____

424. [Avanzado] ¿Cómo probarías un caso de concurrencia o race condition?

Tu respuesta: _____ _____

425. [Avanzado] ¿Qué es mutation testing y qué te puede revelar?

Tu respuesta: _____ _____

11. Clean Code, SOLID y diseño

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

426. [Fundamentos] ¿Qué significa código limpio en términos observables y no estéticos?

Tu respuesta: _____ _____

427. [Fundamentos] ¿Qué señales indican que una clase tiene demasiadas responsabilidades?

Tu respuesta: _____ _____

428. [Fundamentos] ¿Qué es el principio de responsabilidad única (SRP)?

Tu respuesta: _____ _____

429. [Fundamentos] ¿Cómo distinguirías “una responsabilidad” de “un método”?

Tu respuesta: _____ _____

430. [Fundamentos] ¿Qué significa Open/Closed Principle?

Tu respuesta: _____ _____

431. [Fundamentos] ¿Cómo aplicarías Open/Closed sin introducir abstracciones innecesarias?

Tu respuesta: _____ _____

432. [Fundamentos] ¿Qué dice Liskov Substitution Principle?

Tu respuesta: _____ _____

433. [Fundamentos] ¿Qué ejemplo clásico demuestra una violación de LSP?

Tu respuesta: _____ _____

434. [Fundamentos] ¿Qué significa Interface Segregation Principle?

Tu respuesta: _____ _____

435. [Intermedio] ¿Qué significa Dependency Inversion Principle?

Tu respuesta: _____ _____

436. [Intermedio] ¿Cómo se relaciona DIP con constructor injection?

Tu respuesta: _____ _____

437. [Intermedio] ¿Qué diferencia hay entre dependencias de alto nivel y detalles de infraestructura?

Tu respuesta: _____ _____

438. [Intermedio] ¿Qué es acoplamiento y qué es cohesión?

Tu respuesta: _____ _____

439. [Intermedio] ¿Cómo reducirías acoplamiento entre controller y persistencia?

Tu respuesta: _____ _____

440. [Intermedio] ¿Qué hace que un método sea difícil de leer?

Tu respuesta: _____ _____

441. [Intermedio] ¿Cuándo dividir un método en funciones más pequeñas empeora el código?

Tu respuesta: _____ _____

442. [Intermedio] ¿Qué es una abstracción prematura?

Tu respuesta: _____ _____

443. [Avanzado] ¿Qué significa YAGNI?

Tu respuesta: _____ _____

444. [Avanzado] ¿Qué significa DRY y cuándo puede aplicarse de forma equivocada?

Tu respuesta: _____ _____

445. [Avanzado] ¿Qué problema puede aparecer al aplicar DRY a dos conceptos que solo “parecen” iguales?

Tu respuesta: _____ _____

446. [Avanzado] ¿Qué es feature envy?

Tu respuesta: _____ _____

447. [Avanzado] ¿Qué significa “tell, don’t ask” y cómo puede ayudarte en dominio?

Tu respuesta: _____ _____

448. [Avanzado] ¿Cuándo usarías Strategy, Factory, Adapter o Decorator?

Tu respuesta: _____ _____

449. [Avanzado] ¿Cómo decidir si un patrón de diseño realmente aporta valor?

Tu respuesta: _____ _____

450. [Avanzado] ¿Qué hace un anti-corruption layer en una arquitectura?

Tu respuesta: _____ _____

12. Arquitectura, DDD, microservicios y sistemas distribuidos

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

451. [Fundamentos] ¿Qué significa arquitectura por capas y qué responsabilidades deberían existir en cada capa?

Tu respuesta: _____ _____

452. [Fundamentos] ¿Qué problemas puede producir una arquitectura de tres capas mal aplicada?

Tu respuesta: _____ _____

453. [Fundamentos] ¿Qué diferencia hay entre arquitectura hexagonal, clean architecture y onion architecture?

Tu respuesta: _____ _____

454. [Fundamentos] ¿Qué son puertos y adaptadores?

Tu respuesta: _____ _____

455. [Fundamentos] ¿Qué dependencias deberían apuntar hacia el dominio en una arquitectura limpia?

Tu respuesta: _____ _____

456. [Fundamentos] ¿Qué es un bounded context en DDD?

Tu respuesta: _____ _____

457. [Fundamentos] ¿Qué es un aggregate y qué invariantes debería proteger?

Tu respuesta: _____ _____

458. [Fundamentos] ¿Qué es aggregate root?

Tu respuesta: _____ _____

459. [Fundamentos] ¿Qué diferencia hay entre entity y value object en DDD?

Tu respuesta: _____ _____

460. [Fundamentos] ¿Qué es un domain service y cuándo usarlo?

Tu respuesta: _____ _____

461. [Intermedio] ¿Qué es un application service y cómo se diferencia de un domain service?

Tu respuesta: _____ _____

462. [Intermedio] ¿Qué es un repository en DDD y por qué no es lo mismo que un DAO?

Tu respuesta: _____ _____

463. [Intermedio] ¿Qué es anemic domain model y qué problemas puede traer?

Tu respuesta: _____ _____

464. [Intermedio] ¿Qué es ubiquitous language?

Tu respuesta: _____ _____

465. [Intermedio] ¿Cómo identificarías boundaries de dominio en una fintech?

Tu respuesta: _____ _____

466. [Intermedio] ¿Qué diferencia hay entre monolito modular y microservicios?

Tu respuesta: _____ _____

467. [Intermedio] ¿Cuándo un monolito modular puede ser suficiente?

Tu respuesta: _____ _____

468. [Intermedio] ¿Qué problemas operativos introduces al pasar de monolito a microservicios?

Tu respuesta: _____ _____

469. [Intermedio] ¿Qué es service discovery?

Tu respuesta: _____ _____

470. [Intermedio] ¿Qué es API Gateway y qué responsabilidades no debería absorber?

Tu respuesta: _____ _____

471. [Avanzado] ¿Qué significa “distributed transaction” y por qué es difícil?

Tu respuesta: _____ _____

472. [Avanzado] ¿Qué problema intenta resolver el patrón Saga?

Tu respuesta: _____ _____

473. [Avanzado] ¿Qué diferencia hay entre choreography y orchestration en una Saga?

Tu respuesta: _____ _____

474. [Avanzado] ¿Qué es eventual consistency?

Tu respuesta: _____ _____

475. [Avanzado] ¿Cuándo aceptarías consistencia eventual y qué compensaciones diseñarías?

Tu respuesta: _____ _____

476. [Avanzado] ¿Qué es a message broker y qué diferencia hay entre queue y topic?

Tu respuesta: _____ _____

477. [Avanzado] ¿Qué ventajas aporta Kafka frente a una cola tradicional en ciertos escenarios?

Tu respuesta: _____ _____

478. [Avanzado] ¿Qué significa at-least-once delivery?

Tu respuesta: _____ _____

479. [Avanzado] ¿Cómo diseñarías un consumidor idempotente?

Tu respuesta: _____ _____

480. [Avanzado] ¿Qué es the outbox pattern y qué inconsistencia ayuda a evitar?

Tu respuesta: _____ _____

13. Producción: observabilidad, Docker, CI/CD y operación

Objetivo: poder explicar el concepto, justificar decisiones y relacionarlo con un backend real.

481. [Fundamentos] ¿Qué diferencia hay entre logging, metrics y tracing?

Tu respuesta: _____ _____

482. [Fundamentos] ¿Qué hace que un log sea útil para producción?

Tu respuesta: _____ _____

483. [Fundamentos] ¿Qué información incluirías siempre en un log de error de backend?

Tu respuesta: _____ _____

484. [Fundamentos] ¿Qué es structured logging y por qué JSON suele usarse en producción?

Tu respuesta: _____ _____

485. [Fundamentos] ¿Qué es un correlation ID y cómo lo propagarías entre servicios?

Tu respuesta: _____ _____

486. [Fundamentos] ¿Qué es distributed tracing?

Tu respuesta: _____ _____

487. [Fundamentos] ¿Qué son traces, spans y attributes?

Tu respuesta: _____ _____

488. [Intermedio] ¿Qué métricas de una API Spring Boot mirarías para saber si está degradándose?

Tu respuesta: _____ _____

489. [Intermedio] ¿Qué es Micrometer y qué papel cumple en Spring Boot?

Tu respuesta: _____ _____

490. [Intermedio] ¿Qué es Spring Boot Actuator y qué endpoints son especialmente útiles?

Tu respuesta: _____ _____

491. [Intermedio] ¿Qué diferencia hay entre liveness y readiness?

Tu respuesta: _____ _____

492. [Intermedio] ¿Qué es un health check y qué debería evitar hacer?

Tu respuesta: _____ _____

493. [Intermedio] ¿Qué problema resuelve Docker en un backend Java?

Tu respuesta: _____ _____

494. [Avanzado] ¿Qué diferencia hay entre una image y un container?

Tu respuesta: _____ _____

495. [Avanzado] ¿Qué principios usarías para construir una imagen Docker pequeña para Spring Boot?

Tu respuesta: _____ _____

496. [Avanzado] ¿Qué ventajas tiene ejecutar una aplicación Java en un contenedor con un JRE/JDK apropiado?

Tu respuesta: _____ _____

497. [Avanzado] ¿Qué es multi-stage build?

Tu respuesta: _____ _____

498. [Avanzado] ¿Qué variables y secretos deberían inyectarse en runtime en lugar de hornearse en la imagen?

Tu respuesta: _____ _____

499. [Avanzado] ¿Qué es CI y qué debería validar un pipeline para un proyecto Java?

Tu respuesta: _____ _____

500. [Avanzado] ¿Qué es CD y qué pasos automatizarías antes de desplegar a producción?

Tu respuesta: _____ _____

Cómo convertir estas 600 preguntas en aprendizaje

- Responde en voz alta primero. Si puedes definir pero no explicar el mecanismo, todavía no dominas el tema.
- Para cada fallo, escribe una explicación de 3 capas: qué es, cómo funciona internamente y cuándo lo usarías.
- Lleva al código las preguntas de diseño: crea pequeños experimentos en Java/Spring para verificar tus hipótesis.
- Repite las preguntas difíciles una semana después y vuelve a medir qué puedes explicar sin consultar.

Objetivo final

No se trata de saber responder de memoria 600 preguntas. Se trata de que, ante un problema de backend, puedas razonar: qué componente interviene, qué contrato ofrece, qué trade-off existe, qué puede fallar y cómo lo comprobarías.

14. Seguridad: secure coding, OWASP, identidad, APIs, criptografía y DevSecOps

Objetivo: poder detectar riesgos antes de que se conviertan en vulnerabilidades, explicar por qué ocurren y diseñar mitigaciones razonables en un backend real.

01. Secretos, credenciales y configuración segura

501. [Seguridad] ¿Qué datos no deberían subirse nunca al repositorio?

Tu respuesta: _____ _____

502. [Seguridad] ¿Por qué un secreto borrado de un archivo puede seguir expuesto en la historia de Git?

Tu respuesta: _____ _____

503. [Seguridad] ¿Cómo almacenarías credenciales de una base de datos en desarrollo, testing y producción sin hardcodearlas?

Tu respuesta: _____ _____

504. [Seguridad] ¿Qué diferencia hay entre una variable de entorno, un secret manager y un archivo de configuración cifrado?

Tu respuesta: _____ _____

505. [Seguridad] ¿Qué harías si descubres que una API key real fue publicada accidentalmente en GitHub?

Tu respuesta: _____ _____

506. [Seguridad] ¿Qué información sensible evitarías incluir en logs aunque sea útil para depurar?

Tu respuesta: _____ _____

507. [Seguridad] ¿Por qué rotar una credencial comprometida es distinto de simplemente borrarla del código?

Tu respuesta: _____ _____

508. [Seguridad] ¿Cómo diseñarías una política de rotación de secretos para una aplicación backend?

Tu respuesta: _____ _____

509. [Seguridad] ¿Qué riesgos aparecen cuando distintas aplicaciones comparten la misma credencial de base de datos?

Tu respuesta: _____ _____

510. [Seguridad] ¿Cómo verificarías automáticamente en CI que no se estén comiteando secretos o credenciales?

Tu respuesta: _____ _____

02. Contraseñas, autenticación e identidad

511. [Seguridad] ¿Por qué nunca deberías guardar contraseñas en texto plano?

Tu respuesta: _____ _____

512. [Seguridad] ¿Qué diferencia hay entre hashing, cifrado y codificación?

Tu respuesta: _____ _____

513. [Seguridad] ¿Por qué una contraseña debería almacenarse con un algoritmo de password hashing específico y no con SHA-256 directo?

Tu respuesta: _____ _____

514. [Seguridad] ¿Qué función cumplen el salt y el factor de costo en el almacenamiento seguro de contraseñas?

Tu respuesta: _____ _____

515. [Seguridad] ¿Qué propiedades buscarías en un mecanismo moderno de password hashing?

Tu respuesta: _____ _____

516. [Seguridad] ¿Qué amenazas intenta mitigar un sistema de rate limiting sobre el endpoint de login?

Tu respuesta: _____ _____

517. [Seguridad] ¿Qué diferencia hay entre autenticación y autorización y por qué confundirlas genera vulnerabilidades?

Tu respuesta: _____ _____

518. [Seguridad] ¿Qué información debería devolver y qué información debería ocultar un endpoint de login cuando el usuario no existe?

Tu respuesta: _____ _____

519. [Seguridad] ¿Cómo diseñarías recuperación de contraseña sin permitir que un atacante descubra qué cuentas existen?

Tu respuesta: _____ _____

520. [Seguridad] ¿Qué problemas de seguridad puede introducir un sistema de “recordarme” implementado sin analizar el ciclo de vida del token?

Tu respuesta: _____ _____

03. Autorización y control de acceso

521. [Seguridad] ¿Qué significa aplicar el principio de mínimo privilegio a un backend?

Tu respuesta: _____ _____

522. [Seguridad] ¿Por qué comprobar solamente que un usuario está autenticado no significa que pueda acceder a un recurso?

Tu respuesta: _____ _____

523. [Seguridad] ¿Cómo diferenciarías autenticación, autorización basada en roles y autorización basada en recursos?

Tu respuesta: _____ _____

524. [Seguridad] ¿Qué es un IDOR/BOLA y cómo podría aparecer en una API REST?

Tu respuesta: _____ _____

525. [Seguridad] ¿Cómo comprobarías que un usuario solo puede modificar sus propios recursos?

Tu respuesta: _____ _____

526. [Seguridad] ¿Por qué nunca deberías confiar en un userId enviado por el cliente para decidir qué registros puede modificar?

Tu respuesta: _____ _____

527. [Seguridad] ¿Cómo diseñarías permisos cuando una aplicación tiene administradores, usuarios normales y recursos pertenecientes a equipos?

Tu respuesta: _____ _____

528. [Seguridad] ¿Qué riesgos aparecen cuando el frontend oculta una opción pero el backend no valida el permiso?

Tu respuesta: _____ _____

529. [Seguridad] ¿Cómo evitarías que un usuario cambie manualmente un identificador en la URL y acceda al recurso de otra persona?

Tu respuesta: _____ _____

530. [Seguridad] ¿Qué diferencia práctica existe entre RBAC y ABAC y cuándo puede volverse insuficiente un simple sistema de roles?

Tu respuesta: _____ _____

04. Validación de entrada, inyección y serialización

531. [Seguridad] ¿Por qué la validación de entrada debe hacerse en el backend aunque el frontend ya valide los datos?

Tu respuesta: _____ _____

532. [Seguridad] ¿Qué es SQL Injection y por qué los prepared statements ayudan a prevenirla?

Tu respuesta: _____ _____

533. [Seguridad] ¿Por qué concatenar strings para construir una consulta SQL es una mala práctica de seguridad?

Tu respuesta: _____ _____

534. [Seguridad] ¿Qué diferencia hay entre validar por lista permitida y bloquear caracteres considerados peligrosos?

Tu respuesta: _____ _____

535. [Seguridad] ¿Cómo distinguirías validación sintáctica de validación semántica?

Tu respuesta: _____ _____

536. [Seguridad] ¿Qué riesgos puede tener deserializar objetos provenientes de una fuente no confiable?

Tu respuesta: _____ _____

537. [Seguridad] ¿Qué es command injection y en qué situaciones un backend Java podría terminar expuesto a ella?

Tu respuesta: _____ _____

538. [Seguridad] ¿Qué es path traversal y cómo podría afectar a un endpoint que permite descargar archivos?

Tu respuesta: _____ _____

539. [Seguridad] ¿Cómo limitarías el tamaño, tipo y cantidad de archivos subidos por un usuario?

Tu respuesta: _____ _____

540. [Seguridad] ¿Qué problemas de seguridad pueden aparecer cuando aceptas JSON con campos que tu modelo de dominio no debería permitir modificar?

Tu respuesta: _____ _____

05. XSS, CSRF, SSRF, CORS y seguridad web

541. [Seguridad] ¿Qué es XSS y cuál es la diferencia entre reflected XSS, stored XSS y DOM-based XSS?

Tu respuesta: _____ _____

542. [Seguridad] ¿Por qué escapar correctamente la salida puede ser más importante que intentar filtrar palabras peligrosas en la entrada?

Tu respuesta: _____ _____

543. [Seguridad] ¿Qué es CSRF y por qué el riesgo depende de cómo se transporta la credencial de autenticación?

Tu respuesta: _____ _____

544. [Seguridad] ¿Qué diferencia hay entre una cookie HttpOnly y una cookie Secure?

Tu respuesta: _____ _____

545. [Seguridad] ¿Qué función tienen SameSite, Secure y HttpOnly y qué amenaza ayuda a mitigar cada atributo?

Tu respuesta: _____ _____

546. [Seguridad] ¿Qué es CORS y por qué no debe confundirse con un mecanismo general de autenticación?

Tu respuesta: _____ _____

547. [Seguridad] ¿Por qué permitir “Access-Control-Allow-Origin: *” puede ser problemático en determinadas arquitecturas?

Tu respuesta: _____ _____

548. [Seguridad] ¿Qué es SSRF y cómo podría un atacante aprovechar un endpoint que recibe una URL para que el servidor haga una petición?

Tu respuesta: _____ _____

549. [Seguridad] ¿Cómo protegerías un servicio backend que necesita descargar recursos desde URLs proporcionadas por usuarios?

Tu respuesta: _____ _____

550. [Seguridad] ¿Qué función cumple Content Security Policy y cómo encaja en una estrategia de defensa contra XSS?

Tu respuesta: _____ _____

06. Criptografía, transporte y protección de datos

551. [Seguridad] ¿Qué diferencia hay entre cifrado simétrico y asimétrico y para qué usarías cada uno?

Tu respuesta: _____ _____

552. [Seguridad] ¿Qué propiedades de seguridad aporta TLS a una conexión HTTP?

Tu respuesta: _____ _____

553. [Seguridad] ¿Qué problema resuelve la validación del certificado durante una conexión TLS?

Tu respuesta: _____ _____

554. [Seguridad] ¿Por qué “cifrar contraseñas” no es el enfoque correcto para almacenarlas?

Tu respuesta: _____ _____

555. [Seguridad] ¿Qué diferencia hay entre confidencialidad, integridad, autenticidad y no repudio?

Tu respuesta: _____ _____

556. [Seguridad] ¿Qué riesgos tiene inventar tu propio algoritmo criptográfico para proteger datos?

Tu respuesta: _____ _____

557. [Seguridad] ¿Qué es un nonce y por qué algunos algoritmos requieren que sea único?

Tu respuesta: _____ _____

558. [Seguridad] ¿Cuándo usarías un MAC/HMAC y cuándo una firma digital?

Tu respuesta: _____ _____

559. [Seguridad] ¿Cómo decidirías qué datos de una base de datos necesitan cifrado en reposo y cuáles no?

Tu respuesta: _____ _____

560. [Seguridad] ¿Qué problemas pueden aparecer cuando el sistema cifra datos pero pierde el control seguro de las claves de cifrado?

Tu respuesta: _____ _____

07. Seguridad en Java, Spring y Spring Boot

561. [Seguridad] ¿Qué mecanismos de Spring Security utilizarías para separar autenticación de autorización?

Tu respuesta: _____ _____

562. [Seguridad] ¿Qué es el SecurityFilterChain y qué papel cumple en una aplicación Spring Security moderna?

Tu respuesta: _____ _____

563. [Seguridad] ¿Por qué confiar ciegamente en un objeto UserDetails construido a partir de datos del cliente sería peligroso?

Tu respuesta: _____ _____

564. [Seguridad] ¿Qué diferencias de seguridad existen entre sesiones de servidor y tokens como JWT?

Tu respuesta: _____ _____

565. [Seguridad] ¿Qué problemas puede provocar guardar un JWT en un lugar accesible por JavaScript?

Tu respuesta: _____ _____

566. [Seguridad] ¿Qué significa validar la firma, expiración, issuer y audience de un JWT y por qué importa cada una?

Tu respuesta: _____ _____

567. [Seguridad] ¿Por qué un JWT firmado no significa que su contenido sea secreto?

Tu respuesta: _____ _____

568. [Seguridad] ¿Cómo protegerías endpoints administrativos en Spring Boot sin depender de reglas del frontend?

Tu respuesta: _____ _____

569. [Seguridad] ¿Qué riesgos tiene habilitar Actuator sin revisar qué endpoints están expuestos?

Tu respuesta: _____ _____

570. [Seguridad] ¿Qué errores de configuración de CORS, CSRF o gestión de sesiones revisarías antes de llevar una aplicación Spring Boot a producción?

Tu respuesta: _____ _____

08. Dependencias, supply chain, Docker y CI/CD

571. [Seguridad] ¿Por qué una vulnerabilidad en una dependencia transitiva puede afectar tu aplicación aunque nunca hayas escrito ese código?

Tu respuesta: _____ _____

572. [Seguridad] ¿Cómo detectarías dependencias con vulnerabilidades conocidas en un proyecto Maven?

Tu respuesta: _____ _____

573. [Seguridad] ¿Qué diferencia hay entre actualizar una dependencia por una versión nueva y fijar versiones reproducibles?

Tu respuesta: _____ _____

574. [Seguridad] ¿Qué riesgos introduce ejecutar el build de un proyecto con plugins o dependencias de terceros no confiables?

Tu respuesta: _____ _____

575. [Seguridad] ¿Por qué deberías revisar también las dependencias de plugins de Maven y no solamente las dependencias de runtime?

Tu respuesta: _____ _____

576. [Seguridad] ¿Qué es software supply chain security y qué intenta proteger?

Tu respuesta: _____ _____

577. [Seguridad] ¿Qué riesgos de seguridad aparecen al ejecutar un contenedor Docker como root?

Tu respuesta: _____ _____

578. [Seguridad] ¿Qué secretos nunca deberías introducir directamente en una imagen Docker?

Tu respuesta: _____ _____

579. [Seguridad] ¿Cómo impedirías que un pipeline CI/CD imprima secretos en los logs?

Tu respuesta: _____ _____

580. [Seguridad] ¿Qué controles agregarías al pipeline para detectar secretos, dependencias vulnerables, imágenes inseguras y código con fallos de seguridad?

Tu respuesta: _____ _____

09. Errores, logs, monitoreo y respuesta ante incidentes

581. [Seguridad] ¿Por qué devolver stack traces completos al cliente puede ser un problema de seguridad?

Tu respuesta: _____ _____

582. [Seguridad] ¿Qué información debería recibir el cliente ante un error interno y cuál debería quedar solamente en logs?

Tu respuesta: _____ _____

583. [Seguridad] ¿Cómo evitarías que un mensaje de error revele nombres de tablas, rutas internas o versiones de software?

Tu respuesta: _____ _____

584. [Seguridad] ¿Qué diferencia hay entre logging para debugging y logging para seguridad?

Tu respuesta: _____ _____

585. [Seguridad] ¿Qué eventos de seguridad considerarías importantes registrar en un sistema de autenticación?

Tu respuesta: _____ _____

586. [Seguridad] ¿Qué datos sensibles deberían redactarse o enmascararse antes de escribirlos en logs?

Tu respuesta: _____ _____

587. [Seguridad] ¿Cómo detectarías patrones como múltiples intentos fallidos de login o accesos anómalos a recursos?

Tu respuesta: _____ _____

588. [Seguridad] ¿Por qué los logs deberían incluir suficiente contexto para investigar un incidente sin registrar datos innecesarios?

Tu respuesta: _____ _____

589. [Seguridad] ¿Qué harías inmediatamente después de descubrir que una credencial de producción fue comprometida?

Tu respuesta: _____ _____

590. [Seguridad] ¿Qué diferencia hay entre prevenir un incidente, detectarlo y responder a él y por qué necesitas las tres capacidades?

Tu respuesta: _____ _____

10. Threat modeling, privacidad y diseño seguro

591. [Seguridad] ¿Qué es threat modeling y en qué momento del desarrollo debería comenzar?

Tu respuesta: _____ _____

592. [Seguridad] ¿Qué activos, actores, trust boundaries y puntos de entrada identificarías al analizar una API bancaria?

Tu respuesta: _____ _____

593. [Seguridad] ¿Cómo utilizarías STRIDE para buscar amenazas en un sistema backend?

Tu respuesta: _____ _____

594. [Seguridad] ¿Qué significa “secure by design” y cómo cambia la forma de diseñar una funcionalidad?

Tu respuesta: _____ _____

595. [Seguridad] ¿Por qué agregar una capa de seguridad después de terminar el sistema suele ser insuficiente?

Tu respuesta: _____ _____

596. [Seguridad] ¿Cómo aplicarías defensa en profundidad a una API que expone datos sensibles?

Tu respuesta: _____ _____

597. [Seguridad] ¿Qué principio de privacidad aplicarías para no almacenar datos personales que tu aplicación realmente no necesita?

Tu respuesta: _____ _____

598. [Seguridad] ¿Cómo decidirías cuánto tiempo conservar datos sensibles y quién debería poder acceder a ellos?

Tu respuesta: _____ _____

599. [Seguridad] ¿Qué preguntas de seguridad harías antes de conectar tu backend con un proveedor externo mediante API?

Tu respuesta: _____ _____

600. [Seguridad] ¿Qué tendría que demostrar un desarrollador para poder decir que una funcionalidad está razonablemente protegida y qué riesgos seguirían requiriendo revisión especializada?

Tu respuesta: _____ _____